

1 Part 10: State Space Models

2 The *state space representation* of a time series model for $\{Y_t\}$ is given
3 by a pair of equations:

4 The *observation equation* is

5
$$Y_t = F_t X_t + v_t$$

6 and the *state equation* is

7
$$X_t = G_t X_{t-1} + w_t$$

8 where

- 9 • Y_t is the k -dimensional observable
- 10 • X_t is the ℓ -dimensional *state vector*
- 11 • F_t is a k by ℓ matrix
- 12 • G_t is a ℓ by ℓ matrix
- 13 • v_t are iid Normal with mean zero and variance V_t
- 14 • w_t are iid Normal with mean zero and variance W_t

15 It is assumed that w_t and v_t are uncorrelated, i.e., $E(w_t v_t') = 0$.

16 It is often the case that F_t , G_t , W_t , and V_t do not vary with t .

1 The state vector at time t is often thought to be “hidden” or “latent.”

2 The *state equation* is

3
$$\mathbf{X}_t = \mathbf{G}_t \mathbf{X}_{t-1} + \mathbf{w}_t$$

4 so the state is updated at each time step, modelled to be a linear
5 combination of the previous states, plus error ($\mathbf{w}_t$).

6 $\mathbf{X}_t$ can be taken to be the “ideal” observable, while the actual observ-
7 able, $\mathbf{Y}_t$ is related to the current state via the *observation equation*:

8
$$\mathbf{Y}_t = \mathbf{F}_t \mathbf{X}_t + \mathbf{v}_t$$

9 The power of this model is its generality: A wide range of models fit
10 into this framework.

1 Specifying the Initial Distribution

2 To fit these models (done via maximum likelihood), one needs to
3 specify an initial distribution for the state vector. It is typically as-
4 sumed to be normal with mean $\mathbf{m}_0$ and covariance $\mathbf{C}_0$.

5 In some cases this distribution will follow naturally from model as-
6 sumptions (e.g., stationarity), while in others it is just assumed that
7 $\mathbf{m}_0$ is a vector filled with zeros or ones, and $\mathbf{C}_0$ is diagonal with very
8 large entries on the diagonal.

1 **Exercise:** Derive a state space representation for the mean zero AR(2)
 2 model.

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \epsilon_t$$

$$\text{Define } \tilde{X}_t = \begin{bmatrix} Y_t \\ Y_{t-1} \end{bmatrix}$$

$$\tilde{G} = \begin{bmatrix} \phi_1 & \phi_2 \\ 1 & 0 \end{bmatrix}$$

Why?

$$\tilde{G} \tilde{X}_{t-1} = \begin{bmatrix} \phi_1 & \phi_2 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} Y_{t-1} \\ Y_{t-2} \end{bmatrix} = \begin{bmatrix} \phi_1 Y_{t-1} + \phi_2 Y_{t-2} \\ Y_{t-1} \end{bmatrix}$$

$$\tilde{w}_t = \begin{bmatrix} \epsilon_t \\ 0 \end{bmatrix} \quad \tilde{W}_t = \begin{bmatrix} \sigma_\epsilon^2 & 0 \\ 0 & 0 \end{bmatrix}$$

Now,

$$\tilde{G} \tilde{X}_{t-1} + \tilde{w}_t = \begin{bmatrix} \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \epsilon_t \\ Y_{t-1} \end{bmatrix} = \begin{bmatrix} Y_t \\ Y_{t-1} \end{bmatrix} = \tilde{X}_t$$

$$\tilde{F} = \begin{bmatrix} 1 & 0 \end{bmatrix}, \text{ so that } \tilde{F} \tilde{X}_t = Y_t$$

$$\text{and } V_t = 0, \quad \tilde{V} = 0$$

To enforce stationarity of $\{Y_t\}$, note that requires $\{\tilde{X}_t\}$ to be stationary

If assume stationary,

$$\begin{aligned}\underline{m}_0 &= E(\underline{X}_0) = E(\underline{X}_{t+1}) = E(\underline{G}\underline{X}_t + \underline{w}_{t+1}) \\ &= \underline{G} E(\underline{X}_t) \\ &= \underline{G} \underline{m}_0\end{aligned}$$

True when $\underline{m}_0 = \underline{0}$, so choose $\underline{m}_0 = \underline{0}$

$$\begin{aligned}\underline{C}_0 &= V(\underline{X}_0) = V(\underline{X}_{t+1}) = V(\underline{G}\underline{X}_t + \underline{w}_{t+1}) \\ &= V(\underline{G}\underline{X}_t) + V(\underline{w}_{t+1}) \\ &= \underline{G} V(\underline{X}_t) \underline{G}^T + \underline{w} \\ &= \underline{G} \underline{C}_0 \underline{G}^T + \underline{w}\end{aligned}$$

$$\text{vec}(\underline{C}_0) = \text{vec}(\underline{G} \underline{C}_0 \underline{G}^T + \underline{w})$$

$$= \text{vec}(\underline{G} \underline{C}_0 \underline{G}^T) + \text{vec}(\underline{w})$$

$$= (\underline{G} \otimes \underline{G}) \text{vec}(\underline{C}_0) + \text{vec}(\underline{w})$$

$$\text{vec}(\underline{C}_0) = (\underline{I}_4 - \underline{G} \otimes \underline{G})^{-1} \text{vec}(\underline{w})$$

$$\begin{aligned}\text{vec}(ABC) \\ &= (C^T \otimes A) \text{vec}(B)\end{aligned}$$

$$\underline{A} \otimes \underline{B} = \begin{bmatrix} a_{11} \underline{B} & a_{12} \underline{B} & \dots & a_{1n} \underline{B} \\ \vdots & \vdots & & \vdots \\ \vdots & \vdots & & \vdots \\ \vdots & \vdots & & \vdots \end{bmatrix}$$

1 **Exercise:** Derive a state space representation for the mean zero
2 ARMA(1,1) model.

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

13 _____

14 _____

15 _____

16 _____

17 _____

18 _____

19 _____

20 _____

21 _____

Part 10: State Space Models

- 1

- 2

- 3

- 4

- 5

- 6

- 7

- 8

- 9

- 10

- 11

- 12

- 13

- 14

- 15

- 16

- 17

- 18

- 19

- 20

1 Fitting a State Space Model in R

2 There are a range of packages available for fitting state space models
3 in R. We will focus here on the implementation in package `d1m`.

4 The main procedure for fitting the model is `d1mMLE()`.

5 The tricky part to using this procedure is specification of the form of
6 the state space representation. This is done via a “build” function,
7 which takes as input the parameters to be estimated, and returns a
8 list with components `m0`, `C0`, `FF`, `GG`, `V`, `W`.

9 For example, consider the AR(2) model described above. The build
10 function, named `buildAR2()` is shown on the following page.

```

1 # The parameter order is (phi1, phi2, log sigma2)
2 buildAR2 = function(x)
3 {
4   FF = matrix(c(1,0),nrow=1)  $\tilde{F}$ 
5
6   GG = matrix(c(x[1],1,x[2],0),nrow=2)  $\tilde{G} = \begin{bmatrix} \phi_1 & \phi_2 \\ 1 & 0 \end{bmatrix}$ 
7
8   V = 0
9
10  W = matrix(c(exp(x[3]),0,0,0),nrow=2)  $\tilde{W} = \begin{bmatrix} \sigma_\epsilon^2 & 0 \\ 0 & 0 \end{bmatrix}$ 
11
12  m0 = rep(0,2)
13
14  C0 = matrix(solve(diag(4) - GG %x% GG) %*%
15    matrix(W,ncol=1),ncol=2)
16
17  return(list(m0 = m0, C0 = C0, FF = FF, GG = GG,
18    V = V, W = W))
19 }

```

20 Note the following:

- 21 • There are three input parameters, ordered as $\phi_1, \phi_2, \sigma_\epsilon^2$.
- 22 • The parameter σ_ϵ^2 has been transformed so that it is not bounded
- 23 below by zero. This aids with performance of the iterative search
- 24 algorithm.
- 25 • The operator `%x%` calculates the Kronecker product.

1 **R Exercise:** Try the following commands. Comment on the results.

```
2 simseries = arima.sim(list(order=c(2,0,0),  
3     ar=c(0.5,0.1)), n=1000, sd=0.1)  
4 holdfit = dlmMLE(simseries, c(0,0,1), buildAR2)
```

5 (The syntax to `dlmMLE()` requires initial values for the parameters;
6 that is the second argument.)

$$\begin{array}{l} \hat{\phi}_1 = 0.527 \quad \hat{\phi}_2 = 0.088 \quad \log \hat{\sigma}_e^2 = -4.56 \\ \hat{\sigma}_e^2 = 0.01 \end{array}$$

The Kalman Filter

A key appeal of the state space formulation of a time series model is the ease with which standard inference goals can be achieved via a set of recursive algorithms generally referred to as the *Kalman Filter*.

Keep in mind that within this framework it is assumed that the state vector $\mathbf{X}_t$ is of primary interest.

We want to consider three fundamental challenges:

- **Filtering:** Estimate $\mathbf{X}_t$ using $\mathbf{Y}_0, \mathbf{Y}_1, \dots, \mathbf{Y}_t$.
- **Prediction:** Forecast $\mathbf{X}_t$ using $\mathbf{Y}_0, \mathbf{Y}_1, \dots, \mathbf{Y}_{t-1}$.
- **Smoothing:** Estimate $\mathbf{X}_t$ using $\mathbf{Y}_0, \mathbf{Y}_1, \dots, \mathbf{Y}_n$ for $t < n$.

Tsay makes the following analogy to reading a handwritten note:

Filtering is figuring out the word you are reading based on knowledge accumulated from the beginning of the note, predicting is to guess the next word, and smoothing is deciphering a particular word once you have read through the note.
(page 561)

1 Here I present the recursive algorithms needed to address each of the
2 challenges above. The notation is largely taken from Brockwell and
3 Davis (2002). See that text for the derivations.

4 The following notation is needed:

- 5 • Let $P_t(X_i)$ denote the best linear predictor of state X_i given $\mathbf{Y}_0$,
6 $\mathbf{Y}_1, \dots, \mathbf{Y}_t$, for $1 \leq i \leq \ell$. The entries are pulled together into a ℓ
7 by one vector $P_t(\mathbf{X})$.
- 8 • Let $\hat{\mathbf{X}}_t$ denote $P_{t-1}(\mathbf{X}_t)$, the forecast of $\mathbf{X}_t$ based on the informa-
9 tion available at time $t - 1$.
- 10 • Let Ω_t be the error covariance matrix for the prediction at time t ,

11
$$\Omega_t = E[(\mathbf{X}_t - \hat{\mathbf{X}}_t)(\mathbf{X}_t - \hat{\mathbf{X}}_t)'].$$

1 The Prediction Problem

2 The forecasts for the next values of the state vector are given by

3
$$\hat{\mathbf{X}}_{t+1} = \mathbf{G}_t \hat{\mathbf{X}}_t + \Theta_t \Delta_t^{-1} (\mathbf{Y}_t - \mathbf{F}_t \hat{\mathbf{X}}_t)$$

4 with covariance matrices

5
$$\Omega_{t+1} = \mathbf{G}_t \Omega_t \mathbf{G}_t' + \mathbf{W}_t - \Theta_t \Delta_t^{-1} \Theta_t'$$

6 where

7
$$\Delta_t = \mathbf{F}_t \Omega_t \mathbf{F}_t' + \mathbf{V}_t$$

8
$$\Theta_t = \mathbf{G}_t \Omega_t \mathbf{F}_t'$$

9 **Even-More-Technical Note:** The matrix Δ_t need not be invertible:

10 The matrix Δ_t^{-1} can be a *generalized inverse*. See

11 http://en.wikipedia.org/wiki/Generalized_inverse

1 The Filtering and Smoothing Problems

2 The *filtered estimates* are

3
$$P_t(\mathbf{X}_t) = \hat{\mathbf{X}}_t + \Omega_t \mathbf{F}'_t \Delta_t^{-1} (\mathbf{Y}_t - \mathbf{F}_t \hat{\mathbf{X}}_t)$$

4 with covariance matrix

5
$$\Omega_{t|t} = \Omega_t - \Omega_t \mathbf{F}'_t \Delta_t^{-1} \mathbf{F}_t \Omega_t.$$

6 The *smoothed estimates* are

7
$$P_n(\mathbf{X}_t) = P_{n-1}(\mathbf{X}_t) + \Omega_{t,n} \mathbf{F}'_n \Delta_n^{-1} (\mathbf{Y}_n - \mathbf{F}_n \hat{\mathbf{X}}_n)$$

8 with covariance matrix

9
$$\Omega_{t|n} = \Omega_{t|n-1} - \Omega_{t,n} \mathbf{F}'_n \Delta_n^{-1} \mathbf{F}_n \Omega_{1,n}'$$

10 and where

11
$$\Omega_{t,n+1} = \Omega_{t,n} (\mathbf{G}_n - \Theta_n \Delta_n^{-1} \mathbf{F}_n)'$$

12 with $\Omega_{t,t} = \Omega_t$.

13 **Exercise:** What are the smoothed estimates when $t = n$?

14 The same as the filtered estimates at time n

15

16

17

1 **Kalman Filter in R**

2 There are two functions that are part of the package `dlm` that are
3 useful for Kalman Filtering.

4 The first is `dlmFilter()` which can be used for both filtering and
5 prediction.

6 The second is `dlmSmooth()`, used to obtain the smoothed estimates.

7 The syntax will be presented in the context of the following example.

Example: Local Trends Model

This example is an adaptation of Section 11.1 in Tsay.

Here we consider a very simple model for a univariate time series $\{Y_t\}$ called a *local trends model*:

$$Y_t = \mu_t + \epsilon_t$$

where

$$\mu_{t+1} = \alpha\mu_t + \eta_t,$$

ϵ_t is normal with mean zero and variance σ_ϵ^2 and η_t is normal with mean zero and variance σ_η^2 .

Exercise: How is this model different from an AR(1) model?

$$\text{AR}(1) : Y_t = \mu + \phi Y_{t-1} + \epsilon_t$$

In LT model, μ is allowed to evolve with time, but we observe Y_t , i.e. μ_t plus error

1 **Exercise:** Write up this model in state space form.

2 $X_t = \mu_t$

3 $G = \alpha \Rightarrow X_t = G X_{t-1} + w_t$

4 $W = \sigma_w^2$

5 $F = 1$

6 $V = \sigma_e^2 \Rightarrow F X_t + V_t = Y_t$

7

1 The “build” function for this model is quite simple:

```
2 # This function needs to return: m0, C0, FF, GG, V, W
3 # The parameter order is (sigma2e, sigma2eta, alpha)
4                               ^      ^
                               log    log
5 buildLT = function(x)
6 {
7     FF = diag(1)
8
9     GG = diag(1) * x[3]
10
11     V = diag(1) * exp(x[1])
12
13     W = diag(1) * exp(x[2])
14
15     m0 = diag(1)
16
17     C0 = diag(1) * 1e7
18
19     return(list(m0 = m0, C0 = C0, FF = FF, GG = GG,
20               V = V, W = W))
21 }
```

22 Note that the parameters are transformed so that they are unbounded.

1 The application we consider is as follows:

2 The log daily return of an asset can be decomposed

3
$$\text{log return for day } j = \sum_{i=1}^m r_{ij}$$

4 where r_{ij} is the log return over a small time interval, e.g., ten minutes.

5 Then, the *realized volatility* is

6
$$\text{realized volatility for day } j = \sum_{i=1}^m r_{ij}^2$$

7 The distribution of the log of the realized volatility is reasonably ap-
8 proximated by a normal, with mean equal to the log *volatility* of the
9 asset. We will hence employ the local trends model above, with μ_t
10 equal to the (unobservable) log volatility on day t and Y_t is the real-
11 ized volatility on day t .

1 The data we have (from Tsay) are for Alcoa, from January 2, 2003 to
2 May 7, 2004. The data set, named `aa-3rv.txt`, has three columns,
3 for realized volatility based on 1 minute, 10 minute, and 20 minute
4 intervals; we will use the 10 minute data.

5 We assume that the function `buildLT()` is available. Then the model
6 can be fit as follows:

```
7 realvol = read.table("aa-3rv.txt")  
8 rvser = log(realvol[,2])  
9 holdfit = dlmMLE(rvser, c(0,0,1), buildLT)
```

10 We see successful convergence (check that `holdfit$convergence`
11 equals zero), and that the parameter estimates are as follows:

```
12 $par  
13 [1] -1.4702599 -5.1605642 0.9967607
```

$\log(\hat{\sigma}_e^2)$ $\log(\hat{\sigma}_n^2)$ $\hat{\alpha}$

1 **Exercise:** Interpret these parameter estimates.

2 We see that $\hat{\sigma}_h^2 \ll \hat{\sigma}_\epsilon^2$, so the error in
3 the obs. of $\log RV$ is much larger
4 than the day-to-day fluctuations in $\log Vol(\mu_t)$

5
6 Note that $\hat{\alpha} \approx 1$, so μ_t is moving like
7 a random walk
8
9
10
11
12
13
14
15
16
17
18
19
20
21

1 Now consider applying the Kalman Filter. The function `dlmFilter()`
 2 will return the filtered states, and the forecasts.

3 The syntax is

4 `filtout = dlmFilter(rvser, buildLT(holdfit$par))`

5 and then `filtout$m` holds the filtered states, while `filtout$a`
 6 holds the forecasts. See Figure 1.

7 Note that `buildLT(holdpar$par)` takes the estimated parameters
 8 and plugs them into our “build” function to return the fitted forms
 9 for F , G , W , and V . These are then used by `dlmFilter()`.

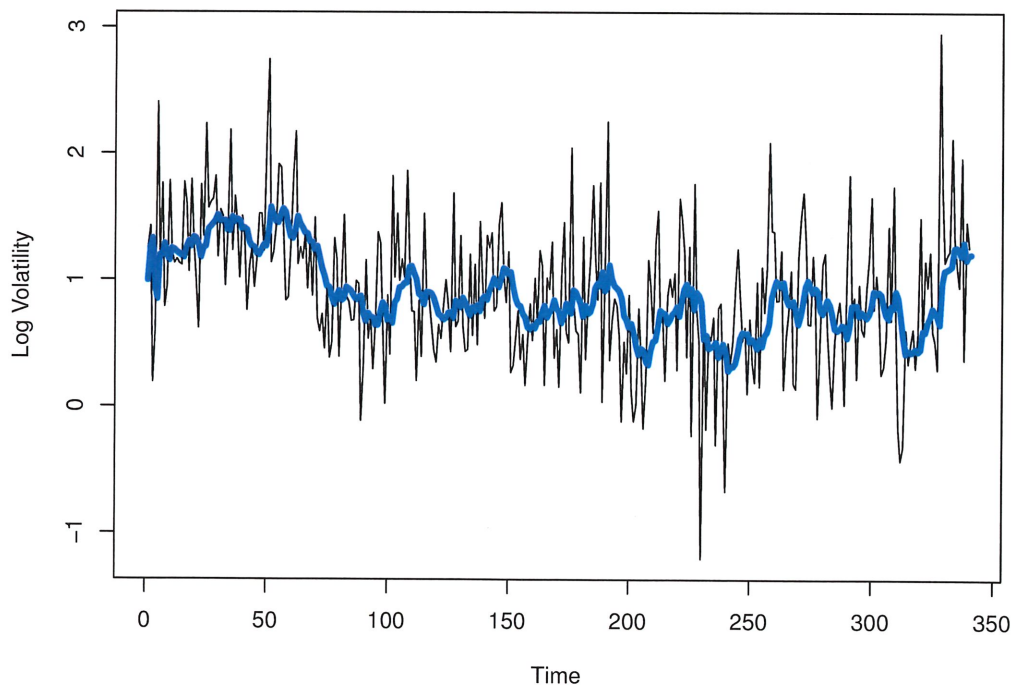


Figure 1: Comparison of the observed series and the filtered states.

- 1 To obtain the smoothed states, use `dlmSmooth()`:
- 2 `smoothout = dlmSmooth(rvser, buildLT(holdfit$par))`
- 3 and find the smoothed version in `smoothout$s`.

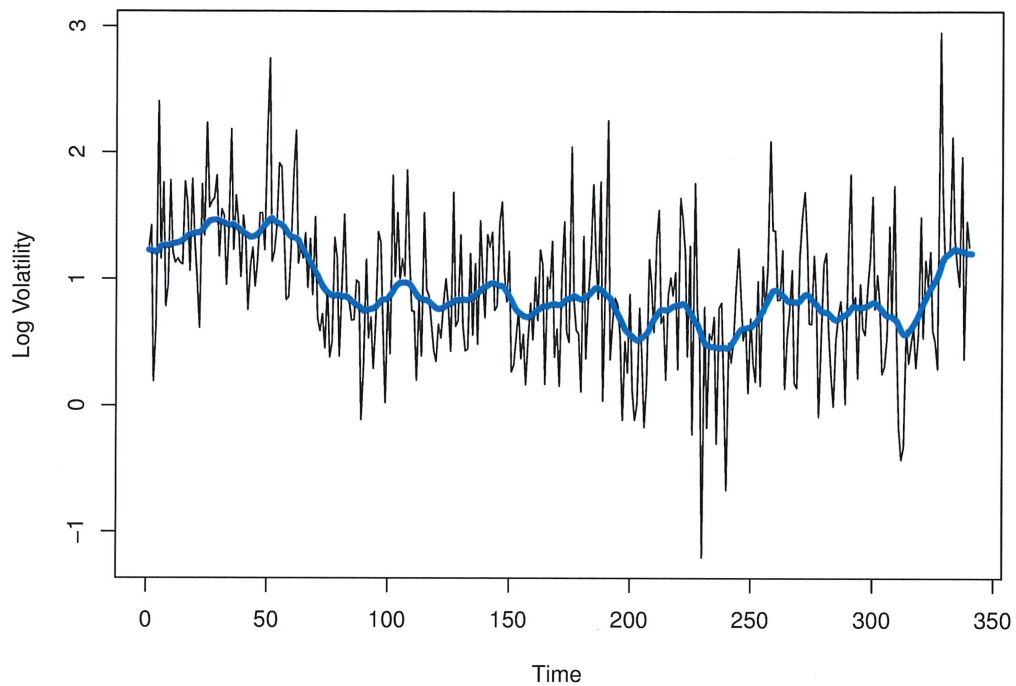


Figure 2: Comparison of the observed series and the smoothed states.

1 **Diagnostics**

2 The output of `dlmFilter()` also includes the one-step forecasts for
3 the observed series Y_t ; these can be found from `holdfit$f`.

4 These allow for the calculations of residuals as the differences be-
5 tween these forecasts and the actual observations.

6 Raw and standardized residuals can be found from

7 `residuals(filtout, type="raw")`

8 `residuals(filtout, type="standardized")`

9 Note that the default is to include the standard deviations of the
10 residuals, so the output is a list consisting of two components named
11 `$res` and `$sd`.

Example: CAPM with Time-Varying Coefficients

This is based on Section 11.3.1 in Tsay.

In the standard CAPM regression model, the response is the excess returns on chosen equity, the single predictor is the excess market returns, and the intercept and slope parameters (α and β) are constant in time.

Here we generalize this to the following model:

$$r_t = \alpha_t + \beta_t r_{M,t} + \epsilon_t$$

$$\alpha_{t+1} = \alpha_t + \eta_t$$

$$\beta_{t+1} = \beta_t + \tau_t$$

where ϵ_t , η_t , and τ_t are assumed to be normal with mean zero and respective variances σ_ϵ^2 , σ_η^2 , and σ_τ^2 . r_t is the excess returns on the equity, while $r_{M,t}$ is the excess returns on the market.

This model has *time-varying coefficients*.

1 **Exercise:** Derive the state space representation for this model.

$$2 \quad \underline{X}_t = \begin{bmatrix} \alpha_t \\ \beta_t \end{bmatrix}$$

$$3 \quad \underline{W}_t = \begin{bmatrix} \sigma_n^2 & 0 \\ 0 & \sigma_x^2 \end{bmatrix} \quad \underline{G} = \underline{I}$$

$$6 \quad \text{So, } \underline{X}_t = \underline{G} \underline{X}_{t-1} + \underline{W}_t$$

$$8 \quad \underline{F}_t = \begin{bmatrix} 1 & \Gamma_{m,t} \end{bmatrix} \quad \text{not constant in time}$$

$$11 \quad \underline{F}_t \underline{X}_t = \alpha_t + \beta_t \Gamma_{m,t}$$

$$12 \quad \underline{V} = \sigma_e^2$$

$$15 \quad \underline{F}_t \underline{X}_t + V_t = \alpha_t + \beta_t \Gamma_{m,t} + \epsilon_t$$

1 Handling Time-Varying Matrices with `dlm`

2 Package `dlm` can handle the case where F (or G , V , or W) is a func-
3 tion of time t . This is done using the arguments `JFF` (or `JGG`, `JV`, or
4 `JW`), along with a new returned matrix named X .

5 These steps should be followed when constructing the “build” func-
6 tion. (I am going to describe this for the case where F varies.)

7 • The function should still return FF , a matrix of the correct dimen-
8 sion. For those entries of F_t which are constant in time, place
9 those constant values in the appropriate positions in FF .

10 • The function should also return `JFF`, another matrix the same di-
11 mension as FF . For those entries of F_t which are constant in time,
12 a value of zero should be entered. For non-constant entries, place
13 a positive integer in that position of `JFF`. That integer specifies
14 which column of matrix X holds the time-varying value of that
15 entry.

$$\text{In ex: } JFF = \begin{bmatrix} 0 & 1 \end{bmatrix}$$

16 • Finally, the function needs to return a matrix named X . The num-
17 ber of rows of X should equal the number of time points, i.e., it
18 should be the same length as the observed series. Each column
19 holds the corresponding time-varying information for `JFF`.

Below is the appropriate “build” function for this model.

```

1  # The parameter order is (sigma2eta, sigma2tau, sigma2eps).
2                                     ^      ^      ^
3                                     log    log    log
4  buildVarCoefReg = function(pars,x)
5  {
6      FF = matrix(c(1,1),nrow=1)
7
8      JFF = matrix(c(0,1),nrow=1)
9
10     X = matrix(x,ncol=1)
11
12     GG = diag(2)
13
14     W = diag(c(exp(pars[1]),exp(pars[2])))
15
16     V = diag(1) * exp(pars[3])
17
18     m0 = c(1,1)
19
20     C0 = diag(2) * 1e7
21
22     return(list(m0 = m0, C0 = C0, FF = FF, GG = GG,
23               V = V, W = W, JFF=JFF, X=X))
24 }

```

1 **Exercise:** Consider the following R commands to obtain data and fit
2 the varying coefficient model described above. Run these commands
3 and inspect the filtered and smoothed states. Also inspect the resid-
4 uals.

```
5 library(quantmod)
6 source("http://www.stat.cmu.edu/~cschafer/MSCF/getFamaFrench.txt")
7
8 PNC = getSymbols("PNC", from="2018-1-1",
9   to="2018-6-30", auto.assign=F)
10
11 ffhold = getFamaFrench(from = "2018-1-1",
12   to="2018-6-30")
13
14 ffhold$PNCexret = 100*dailyReturn(PNC) - ffhold$RF
15
16 holdfit = dlmMLE(ffhold$PNCexret, c(0,0,0),
17   buildVarCoefReg, x=ffhold$Mkt.RF)
18
19 filtout = dlmFilter(ffhold$PNCexret,
20   buildVarCoefReg(holdfit$par, ffhold$Mkt.RF))
21 smoothout = dlmSmooth(ffhold$PNCexret,
22   buildVarCoefReg(holdfit$par, ffhold$Mkt.RF))
```

Example: A Stochastic Volatility Model

This is largely based on Section 11.4 in Chan (2010), *Time Series*

A basic *stochastic volatility* model for the series $\{X_t\}$ can be written as follows:

$$X_t = \sigma_t \epsilon_t$$

$$\sigma_t = \exp(\psi_t/2)$$

where $\{\psi_t\}$ is ARMA(p, q) and the ϵ_t are iid standard normal.

Exercise: In what fundamental way is this model different from the GARCH models we discussed?

σ_t is a random function of the past

For GARCH, given info to time $t-1$,
know the value of σ_t

1 Note that $\psi_t = \log(\sigma_t^2)$. Let $Y_t = \log(X_t^2)$ and $U_t = \log(\epsilon_t^2)$. $X_t^2 = \sigma_t^2 \epsilon_t^2$
2 $\log X_t^2 = \log \sigma_t^2 + \log \epsilon_t^2$

2 Then, by squaring each side and taking the log, our model can be
3 rewritten as

4
$$Y_t = \psi_t + U_t$$

5 and let's just assume that $\{\psi_t\}$ is AR(1), so that

6
$$\psi_{t+1} = \alpha\psi_t + \tau_t,$$

7 although a more general ARMA model can be used.

8 The challenge to fitting this model is that U_t is not normally dis-
9 tributed. In fact, it has mean of approximately -1.27 and variance
10 $\pi^2/2$.

1 The Quasi-Maximum Likelihood (QML) Estimator, proposed by Har-
2 vey, Ruiz, and Shephard¹ proceeds with the standard state space
3 model with observation equation

$$4 \quad Y_t = -1.27 + \psi_t + U'_t$$

5 This model assumes U'_t is normal with mean zero when that is only
6 a rough approximation.

7 While this is a reasonable approximation (Harvey, et al. state that
8 “experiments . . . suggest that QML method works well for the sam-
9 ple sizes typically encountered in financial economics . . .”), many al-
10 ternative approaches to estimating this model have been proposed.

¹*Review of Economic Studies*, **61**, 1994

Exercise: Derive the state space form for this model.

$$\tilde{F} = \begin{bmatrix} -1.27 & 1 \end{bmatrix}$$

$$\tilde{X}_t = \begin{bmatrix} 1 \\ \psi_t \end{bmatrix} \quad \tilde{V} = [V(u_t)] = \frac{\pi^2}{2}$$

$$\tilde{G} = \begin{bmatrix} 1 & 0 \\ 0 & \alpha \end{bmatrix}$$

$$\tilde{G} \tilde{X}_t = \begin{bmatrix} 1 \\ \alpha \psi_{t+m} \end{bmatrix}$$

$$\tilde{W} = \begin{bmatrix} 0 & 0 \\ 0 & \sigma_x^2 \end{bmatrix}$$

1 Below is the “build” function for this case.

```
2 # The parameter order is (alpha, sigma2tau)
3
4 buildSV = function(x)
5 {
6   FF = matrix(c(-1.27, 1), nrow=1)
7
8   GG = matrix(c(1, x[1]), nrow=1)
9         matrix(c(1, 0, 0, x[1]), nrow=2)
10  V = diag(1) * pi^2/2
11
12  W = matrix(c(0, 0, 0, exp(x[2])), nrow=2)
13
14  m0 = matrix(c(1, 1), nrow=1)
15
16  C0 = diag(c(0, 1e7))
17
18  return(list(m0 = m0, C0 = C0, FF = FF, GG = GG,
19             V = V, W = W))
20 }
```

- 1 **R Exercise:** Try fitting a model of this form using the commands
2 below. Comment on the results.

```
3 library(quantmod)
4
5 IBM = getSymbols("IBM",from="2017-1-1",to="2017-12-31",
6   auto.assign=F)
7 holdser = log(dailyReturn(Ad(IBM))^2)
8 holdser = holdser[-1]
9 holdser = holdser[!is.infinite(holdser)]
10
11 holdfit = dlmMLE(holdser,c(0,0),buildSV)
```

$$\hat{\alpha} = \cancel{0.00189} - 0.00195$$

$$\hat{\sigma}_{\pi}^2 = \cancel{0.00872} = 0.00382$$

Generalizations

The state space model is often written as follows:

The *observation equation* is

$$\mathbf{Y}_t = \mathbf{c}_t + \mathbf{F}_t \mathbf{X}_t + \mathbf{v}_t$$

and the *state equation* is

$$\mathbf{X}_{t+1} = \mathbf{d}_t + \mathbf{G}_t \mathbf{X}_t + \mathbf{w}_t,$$

where $\mathbf{c}_t$ and $\mathbf{d}_t$ are n by one and ℓ by one vectors.

This can be taken a step further, and the state equation can include a term $\mathbf{M}_t \mathbf{U}_t$, where $\mathbf{U}_t$ is a matrix of deterministic predictors, and $\mathbf{M}_t$ is a matrix of regression-like coefficients.

I am not aware of implementations of these models in R.

The R package `KFAS` includes an implementation that allows for the observation Y_t to have distributions other than normal, i.e., Poisson, binomial, gamma, and negative binomial